LPPool2d#

https://pytorch.org/docs/stable/generated/torch.nn.modules.pooling.LPPool2d.

Description:

Applies a 2D power-average pooling over an input signal composed of several input planes. On each window, the function computed is: At p = ∞\infty∞, one gets Max Pooling At p = 1, one gets Sum Pooling (which is proportional to average pooling) The parameters kernel_size, stride can either be:

Function Signature

```
class torch.nn.modules.pooling.LPPool2d(norm_type,
kernel_size, stride=None, ceil_mode=False)[source]#
Parameters
Shape:
forward(input)[source]#
Return type
```

Returns:

Tensor

Code Examples

```
>>> # power-2 pool of square window of size=3, stride=2
>>> m = nn.LPPool2d(2, 3, stride=2)
>>> # pool of non-square window of power 1.2
>>> m = nn.LPPool2d(1.2, (3, 2), stride=(2, 1))
>>> input = torch.randn(20, 16, 50, 32)
>>> output = m(input)
```

Notes & Warnings

NOTE Note If the sum to the power of p is zero, the gradient of this function is not defined. This implementation will set the gradient to zero in this case.

Detailed Documentation

Documentation

Applies a 2D power-average pooling over an input signal composed of several input planes.

On each window, the function computed is:

At p = ∞\infty∞, one gets Max Pooling

At p = 1, one gets Sum Pooling (which is proportional to average pooling)

The parameters kernel_size, stride can either be:

a single int – in which case the same value is used for the height and width dimension

a tuple of two ints – in which case, the first int is used for the height dimension, and the second int for the width dimension

If the sum to the power of p is zero, the gradient of this function is not defined. This implementation will set the gradient to zero in this case.

kernel_size (Union[int, tuple[int, int]]) – the size of the window

stride (Union[int, tuple[int, int]]) – the stride of the window. Default value is kernel_size

ceil_mode (bool) – when True, will use ceil instead of floor to compute the output shape

Input: (N,C,Hin,Win)(N, C, H_{in}, W_{in})(N,C,Hin,Win) or (C,Hin,Win)(C, H_{in}, W_{in})(C,Hin,Win).

Output: (N,C,Hout,Wout)(N, C, H_{out}, W_{out})(N,C,Hout,Wout) or (C,Hout,Wout)(C, H_{out}, W_{out})(C,Hout,Wout), where

Runs the forward pass.